Informe post-incidente — Simulacro de restauración de Food Store

TP Unidad 6 · Bases de Datos II · UTN — Ignacio Martín Rodríguez — 20/10/2026 · PostgreSQL 16.15 · rama **tp-unidad-6**, carpeta **food-store-tp-u6/**

1. Respaldo utilizado y tiempos del simulacro

Dato	Valor	Evidencia
Respaldo	**foodstore_20261020_084525.dump** — tomado el **20/10/2026 08:45:25 (-03)**; formato custom; 2.800.370 bytes; SHA-256 83fcb30a…6ac90fb; pg_dump en 0,6 s. Conteos y sumas de control tomados en el mismo snapshot (pg_export_snapshot).	01_backup_ok.txt
Base de la prueba	Restaurada en **foodstore_restore_test**. **foodstore** no se modificó (147.044 pedidos antes y después).	05, 06
Cronómetro	Inicio 08:48:58,926 (antes de createdb) → fin 08:49:00,712 (validaciones OK). createdb 76 ms · pg_restore 1.027 ms · ANALYZE 231 ms · validaciones 433 ms. **Total de restauración y validación: 1,8 s** (1,9 s contando la verificación previa del SHA-256).	05_simulacro_restore.log

2. RPO y RTO: previstos y observados

	Qué mide	Previsto (SPEC_U6)	Observado
RPO	Cuántos datos se pueden perder: el tiempo entre el último respaldo y el incidente.	≤ 24 h (respaldo diario a las 03:10)	**3,5 min** (antigüedad del respaldo al iniciar). Se perderían **40 pedidos nuevos y 3.000 cancelaciones** hechas después del respaldo. El 3,5 min es del laboratorio: con el plan diario, el peor caso real es de casi 24 h de ventas.
RTO	Cuánto se tarda en volver a tener la base operativa y validada.	≤ 15 min	**1,8 s** (mediana de 5 repeticiones: 1,87 s). Muy por debajo del objetivo para 2,7 MB comprimidos; crece con el tamaño de la base.

3. Resultado de las validaciones

Control	Resultado
Filas por tabla contra el respaldo	categoria 15 · cliente 5.003 · producto 307 · pedido 150.004 · pedido_producto 375.006 — **5/5 iguales**
Restricciones lógicas	Productos con precio o stock negativo: **0** · líneas con cantidad ≤ 0 o precio < 0: **0** · líneas sin pedido: **0**
Sumas de control	Stock total, importe total de líneas e id máximo de pedido: **3/3 iguales**. Los 2 disparadores de negocio (RN1–RN3) se restauraron.
Pruebas de falla	Respaldo: servidor caído, sin credenciales y pg_dump fallido → código 1, sin .dump (3/3). Restauración: sin --recrear, respaldo dañado y truncado → código 1 (3/3).

4. Problemas reales encontrados

a) La consulta de validación de la consigna usa **detalle_pedido**, que no existe en el esquema de Food Store: corrida tal cual da relation "detalle_pedido" does not exist (06_validacion_consigna.txt). Se adaptó a **pedido_producto**. **b)** El comando de referencia de pg_dump fue rechazado por la regla final reject de pg_hba.conf agregada en la U5 (00_primer_intento_pg_dump.txt). Se habilitó SCRAM por TCP sólo para postgres y las tres bases del TP, con la contraseña en ~/.pgpass. **c)** En la primera prueba, backup_foodstore.sh quedó colgado al cerrar la sesión del snapshot (faltaba \q) y, al interrumpirlo, dejó un .dump.parcial: se corrigió y se agregó una limpieza con trap.

5. Mejora para la próxima ejecución

Tiempo: restaurar con **pg_restore --jobs** igual a la cantidad de CPU. Medido con 5 corridas alternadas en 2 CPU: la mediana de pg_restore bajó de 1.066 a 831 ms (−22 %) y la del RTO de 1,87 a 1,64 s (−13 %) (08_mejora_jobs.txt). La ganancia crece con el tamaño de la base. **Riesgo:** el punto débil es el RPO de hasta 24 h. Se propone activar el archivado de WAL (recuperación a un punto en el tiempo) o, como mínimo, un segundo respaldo al cierre del turno noche.

6. Trabajo asistido por IA y cómo se verificó

Los scripts y el SQL los generó un asistente de IA (**Claude**, usado en lugar de OpenCode) a partir de SPEC_U6.md y de las reglas de AGENTS.md. Nada se aceptó sin ejecutarlo contra la base: cada script se corrió a mano y se leyó su salida. Además, se escribieron pruebas de falla para el respaldo y la restauración, y se compararon los conteos contra un snapshot. Las pruebas detectaron los errores del punto 4c. Se comprobó también que la contraseña no aparece en el log del servidor ni en el repositorio. VACUUM FULL quedó excluido: maintenance.sql sólo ejecutó ANALYZE categoria, porque nunca había tenido estadísticas; ninguna tabla superaba el umbral de VACUUM.